

JitterNot: LSTM-based Model Predictive Controller for Adaptive Video Streaming

CS293N – ML for Networked Systems
Spring 2022 Final Project Report

Satyam Awasthi Shubham Talbar Nikunj Baid

1 Abstract

Video conferencing has become an essential part of everyday life worldwide. Since the COVID-19 pandemic, Video Conferencing Application (VCA)s have become quintessential in the space of Education, Business, and Telehealth. How a VCA varies video quality under different network conditions is pivotal in determining the user-perceived Quality of Experience (QoE). Adaptive Bitrate (ABR) algorithms are often used to optimize the QoE during video conferencing. In the client-side VCA, the predicted throughput and a video codec act together to improve the user's QoE. They use fixed control rules or heuristics and so suffer from a key limitation: they don't take into account the deployment environment and network conditions. We use the Model Predictive Controller (MPC) in a closed-loop and supply it with the predicted throughput to effectively evaluate QoE parameters. Our contributions are to use (i) an Long Short-Term Memory (LSTM) model for throughput prediction that simplifies the control system from a Multi-Input Single-Output (MISO) to a Single-Input Single-Output (SISO), and (ii) the MPC controller that optimizes the perceived QoE. We demonstrate that the closed-loop control by JitterNot can improve or match the default codec's QoE during a video conference.

2 Introduction

Due to the pandemic, our world was transformed and the use of VCAs has been encouraged worldwide across a majority of spheres affecting the lives of all including Education, the Corporate Sector, Medicine, and Telehealth. User-perceived QoE is thus critical for the performance of a VCA and could have a profound impact on any of these fields. Issues such as stuttering, and low video quality at ample bandwidth can be detrimental and lead to a substandard user experience. Considering that there is little support in the network for optimizing QoE, bottlenecks could occur anywhere in the delivery system. Thus, a robust bitrate adaptation algorithm on the client-side is needed to ensure a good user experience. ABR for VCAs fundamentally differs from the ABRs used by

leading video streaming services like Netflix [6] as these are built around the liberty of utilizing buffer occupancy. The widely used video conferencing systems today include FaceTime, Hangouts, Skype, and WebRTC. These applications are used for person-to-person videoconferencing, cloud video-gaming, teleoperation of robots and vehicles, and any setting where video must be encoded and sent with low latency over the network. The standard approach used in the VCAs today involves two components: a transport protocol and a video codec [4]. During a video call, the transport protocol manages the transmission of the video stream packets to the receiver and processes acknowledgments, and congestion signals to estimate the average data rate of the network path. It supplies this estimate to the codec, a distinct module with its own internal control loop. The codec selects encoding parameters (a frame rate and quality setting like the Quantization Parameter (QP)) and generates a compressed video stream with an average bitrate that approximates the estimated network capacity. We use Google Meet as the VCA for all our experimentations and develop the closed-loop control for optimizing QoE, JitterNot, to be applied on top of the internal controls in place in Google Meet. The results in section 5 indicate that JitterNot performs at least as well as the existing ABR mechanism of Google Meet outperforming it intermittently by our QoE definition. In general, we summarize our contributions as follows:

- To the best of our knowledge, JitterNot is a one of its kind system to utilize Deep Learning (DL) techniques to tap the potential for MPC-based ABR algorithm for video transmission and is the first to do so for video conferencing.
- We develop an effective throughput predictor using an LSTM model to predict the throughput for future windows. Further, we also model QP, a valuable QoE metric using Machine Learning (ML) methods.
- We curated a dataset for training our models using MahiMahi [9] to emulate different network conditions.

This paper proceeds as follows. Section 3 discusses background information on the real-time video system of Google Meet and related work including work on MPC. We describe the design and implementation of JitterNot in Section 4 where we describe the data collection process, the VCA as a control system, model it, and design the MPC controller. Section 5 presents the results of the evaluation. We discuss the limitations of the system and its evaluation in Section 6. JitterNot is publicly available, and the experiments reported in this paper are intended to be reproducible. The source code and raw data from the evaluation are available at github repository [13]

3 Related Work

WebRTC’s reference implementation has been incorporated into major Web browsers [4]. We conducted our sessions on Google Meet (which uses WebRTC [10]) and used the Google Chrome browser. When using Google Chrome, data from both the sending

and receiving parties in a WebRTC-based telemeeting can be gathered via the WebRTC internals page (<chrome://webrtc-internals/>). In our work, we examine two-party calls wherein one peer (sender A) will “share screen” to stream a video being played on a local video player. While the receiver (peer B) is muted and also has his camera turned off.



Figure 1: A two-party video conference call opens four tracks

Performance Statistics in WebRTC-based Video Communication [5] explain the peer connection between peers A and B. A unique Synchronization Source (SSRC) ID (one per track) is shared between them. Peer A (sender), has stats labeled as sent (e.g., bitsSentPerSecond) whereas peer B(receiver), has stats labeled as received (e.g., bitsReceivedPerSecond). We will follow the convention of referring to a sender peer as A and the receiver as B in our report. For data collection **Section 4.1**, we vary the network conditions for A, keeping the conditions for B unrestricted.

MDN Web Docs [12] gives us necessary details about the `RTCStatsReport` of a `mediaStream` captured using the WebRTC API: for A it includes a `remote-outbound-Real-time Transport Protocol (RTP) statistics object` (of type `RTCRemoteOutboundRtpStreamStats`); it should also have a corresponding `inbound-RTP` object. Both of these provide information about the same batch of packets being transmitted from the remote peer to the local device. The difference is that `remote-outbound-RTP` describes statistics about the transmission(s) from the perspective of the remote peer (and so includes mostly empty or null values), while `inbound-RTP` offers statistics about the incoming data from the local peer’s perspective. Thus, the `inbound-RTP` provides the feedback of video quality as perceived at the receiver B.

2prong, an ABR solution with Deep Neural Network (DNN) and MPC for video streaming [7] explored the design of ABR algorithms for Mobile network video streaming through Bayesian model predictive control-based approach 2prong. In contrast to state-of-art adaptation algorithms, 2prong takes Uncertainty-Aware Robust Adaptive video streaming transmission. It utilizes high-dimensional contextual information such as buffer occupancy, predicted throughput, and video quality to find the most valuable information for quality adaption in real-time. [7] demonstrated that 2prong can improve the video quality of video streaming transmission. Thus, it demonstrates that using a DNN for MPC controller for video streaming is relevant and motivates us to apply a similar approach for VCAs.

Artificial Neural Network (ANN)-based model predictive control (MPC)

and optimization of Heating, Ventilation, and Air Conditioning (HVAC) systems [11] is another work that uses neural networks based MPC but applies it to HVAC systems. It states that traditionally, the industry has been very reluctant to adopt complicated methods in modeling and control of HVAC systems and due to this reason the majority of HVAC systems are still using very simple on/off or Proportional-Integral-Derivative (PID) controllers instead of more modern controllers such as ANN-based MPC. We see similar skepticism in the adoption of any DNN or blackbox solutions in video conferencing. We take note of the important implementation specifics for the MPC technique and its controller architecture in [11].

Deepmpc: A Mixture ABR Approach via DL and MPC [8] shows that the fusion of DL and MPC is not only more effective and interpretable but also achieves state-of-the-art performance compared with existing algorithms. DeepMPC enhances MPC via two deep learning-based modules, i.e., DL-based Throughput Predictor (DTP), which can precisely predict future bandwidth, and Discounted Factor Optimizer (DFO), which estimates the prediction error. Moreover, its implementation in real-world network environments and experimental results demonstrate the superiority of DeepMPC against existing state-of-the-art approaches. Like [7], [8] is also an ABR solution for video streaming and utilizes buffer occupancy.

4 JitterNot Design

4.1 Collecting data

In order to develop a data-driven model such as RNN or LSTM, a rich dataset is required that captures most of the network conditions. However, tackling a novel problem like ours (designing an ABR framework for VCAs) has the downside of no existing dataset that is publicly available. Thus, the dataset had to be curated independently with a provision for fine adjustment of the network link parameters to emulate different network conditions. Also, the neural network(s) we develop should be relatively uncomplicated for the model to fit the training data well.

MahiMahi’s [9] network emulation tools can be used to emulate many different link conditions. MahiMahi supports emulating fixed propagation delays (DelayShell), fixed and variable link rates (LinkShell), and stochastic packet loss (LossShell). LinkShell also supports various queueing disciplines such as DropTail, DropHead, and active queue management schemes like CoDel. Each of MahiMahi’s network emulation tools can be arbitrarily nested within one another, providing more experimental flexibility. Hence, MahiMahi was chosen as the tool for emulating various network conditions.

The webrtc-internals functionality for Google Chrome has been used by WebRTC application developers to understand the features and functions of their WebRTC services. It is relatively new to explore and utilize these stats to study the QoE aspects of WebRTC services. It enables observation of the performance of the WebRTC connections locally in the browser. During a 10-day test period, we set up video conference sessions on Google Meet, a WebRTC application, in the Google Chrome browser running in the

link, delay, and loss shells - running as nested instances to provision varying network conditions. A mix of network conditions was applied - latency from 0 to 1500ms, link packet loss from 0 to 35%, and link speed from 3 Mbps to 100mbps. Each video session was kept up for 3-4 minutes giving the total volume of the video conference statistics dataset amounting to 6-7 hours.

The sampling resolution of the webRTC internals tool is 1 second. Thereby, we got over 25,000 data points each containing features giving various performance details of the video conference. Also, for each session, the same video was played in a video player offline in the test machines. This helped us to produce accurate results for two reasons: (i) the network in the test machine had only the video conference traffic and not the traffic from streaming the video online, and (ii) video segments with high movement scenes are likely to be encoded with a high bitrate before transmission, effectively having a Variable Bitrate (VBR). Having the same video played throughout the training data would ensure a good fit in our neural network models as each session will have a similar VBR. Although this might not be ideal to generalize for other videos, we expect our MPC controller to make precise decisions for high QoE as it is supplied with feedback. Also, this could be addressed by adding to the data set and using more real-world scenarios during a video conference.

4.2 System Modeling

In this section, we present an outline of JitterNot (Figure 2) that helps describe the ongoing video conference process, running our QoE optimization solution, as a closed-loop controlled system in the time (t) domain. Also, we emphasize explaining in detail the steps for modeling the system and its components. Section 4.4.2 will cover the specifics for the controller.

Figure 2 highlights 4 primary components of the system:

- **WebRTC Stats:** The Google Meet VCA, uses WebRTC, an open framework for the web that enables Real-Time Communications in the browser. The WebRTC Application Programming Interface (API) allows capturing the mediaStream statistics including all the features we need.
- **Throughput Prediction:** Uses an LSTM model to predict the throughput values (in packets/s) for the horizon size of 5 seconds.
- **MPC Controller:** Includes 2 models for QoE estimate (i) frame jitter, which applies a formula to incoming data to obtain the estimate of the frame jitter component of the QoE, and (ii) QP, a polynomial regression model of the quantization parameter, another component of our QoE. It solves the QoE optimization problem using a constraint solver.
- **Video Resolution Switching:** It receives the resolution values that optimize the QoE. And changes the resolution of the ongoing video call.

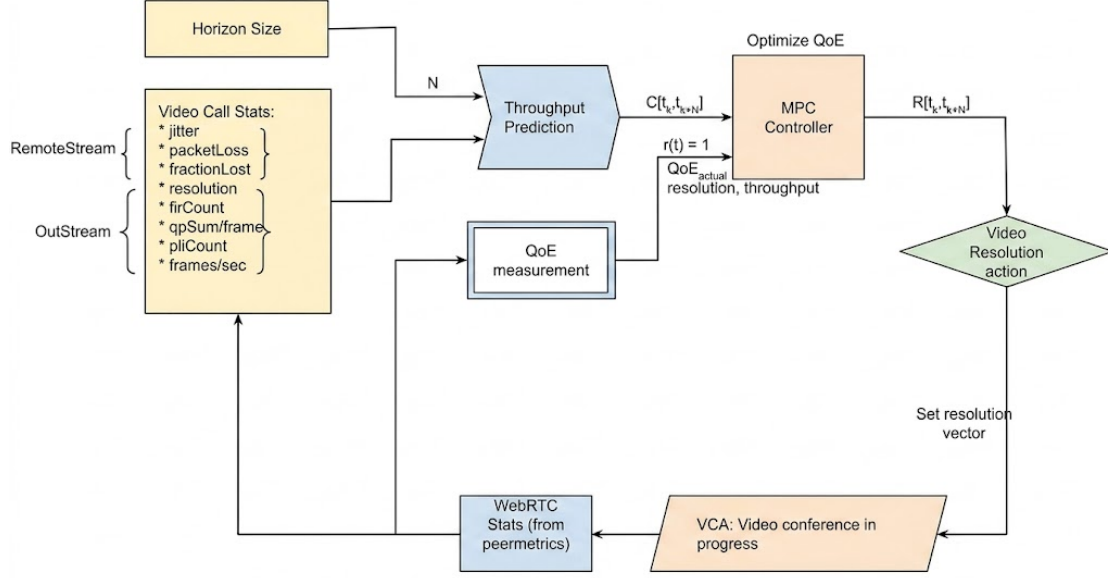


Figure 2: Block Diagram for JitterNot

4.2.1 Feature Selection

One of the major challenges that researchers and machine learning engineers come across is the analysis of high-dimensional data. Feature selection provides an effective way to reduce the dimensionality by getting rid of redundant and irrelevant data. This can also facilitate better accuracy and enhances the explainability of the model while reducing its complexity. Feature selection essentially means selecting a subset of most important features to train the model at hand. This also helps reduce training time and avoid overfitting. For the given problem statement, we used an extra-trees classifier. This class implements a meta-estimator that fits a number of randomized decision trees (a.k.a. extra trees), on various subsamples of the dataset and uses averaging to improve the predictive accuracy and control overfitting. Figure 3 below highlights the top six features with the highest scores that can be used to get a subset of the data for the model to train on.

4.2.2 LSTM model for throughput prediction

Time series prediction is one of the more difficult types of predictive modeling problems. The dependency on sequence among the input variables adds to its complexity. Recurrent Neural Network (RNN)s are designed to handle such dependencies. The LSTM, is a type of RNN that can successfully train relatively large architectures. Another benefit of LSTMs is that in addition to learning long sequences, it can also learn to make a one-shot multi-step forecast which may be useful for time-series forecasting.

For our use case, we train the model to look back ten seconds in time (ten data

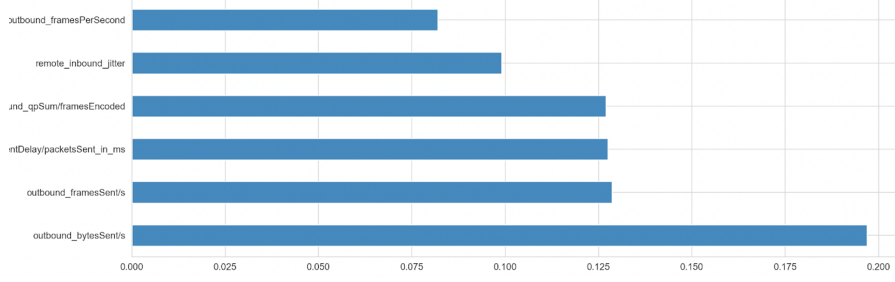


Figure 3: Top six features ranked by score

points), to predict the throughput for the next three seconds, on the dataset accumulated using WebRTC on the Google Meet VCA call.

4.2.3 AutoML for throughput prediction

We also explored the efficacy of an Automated Machine Learning (AutoML) framework in our use case. AutoML frameworks facilitate data cleaning, feature selection, model family selection, hyperparameter tuning, and so on. FEDOT was the framework of choice. It is a framework for automated modeling and ML, aimed to create complex models to obtain an efficient solution for various applied problems.

The model suggested by the FEDOT framework was lagged | Dtreg (Decision Tree Regressor). Although our work utilizes LSTM for throughput prediction, we compare it with the predictions made by the Dtreg. Lagged version of the features allows varying amounts of recent history to be brought into the forecast. This is important to predict the future at time t based on whatever happened until time $t - 1$. Dtreg, on the other hand, can help build regression trees where the target variable being predicted is continuous.

4.3 QoE Metric

There exists a significant variance in user preferences for video streaming QoE. We consider a variety of QoE factors. We use the general QoE metric used by MPC [14] and modify it for relevance in VCA applications; it is given as:

$$\text{QoE}_k = \alpha q(R_k) - \beta P_k - \gamma F_k \quad \text{and normalized } \widehat{\text{QoE}} = \frac{\text{QoE}}{\text{QoE}_{\text{avg}}} \quad (1)$$

Where α, β, γ are constants that normalize the QoE terms $q(R)$, P , and F . For a video conference at time k : R_k is the resolution of the video, $q(R_k)$ maps that resolution to the quality perceived by a user, P_k is qpSum/sec or QP at k . F_k is the frame jitter term, representing the average delay variation for downloading a frame in the past 5 seconds, i.e., for the time window $[k - 4, k]$. We consider the choice of $q(R_n) = \log(R/R_{\min})$. This metric captures the notion that, for some users, the marginal improvement in perceived quality decreases at higher bitrates. The average

and minimum values for any term are calculated from the past 10-second window. We define the individual terms in our QoE metric in detail.

4.3.1 Resolution

Adobe defines video resolution as the number of pixels contained in each frame [1]. Video resolution determines the amount of detail in your video, or how realistic and clear the video appears. It is measured by the number of pixels contained in a frame. A higher number of pixels indicates a higher resolution, and a lower number of pixels makes for a low-resolution video. We define resolution R_k to be a product of frame width and frame height at time k . It captures the amount of data or pixels in each frame so it is proportional to the frame size. Moreover, we will use R_k as the manipulated variable for our MPC controller.

4.3.2 Frame Jitter Estimation

Jitter is the amount of variation in packet delay, which is why it is also frequently called delay variation. It is measured by calculating the average variation in packet arrival times at the receiving node occurring over a fixed interval. At callstats.io, the jitter calculation interval varies between five and ten seconds depending on endpoint configuration [15]. Frame jitter can thus be derived from network jitter, by modifying the definition to measure delay variations for each frame instead of a single packet and so it varies with the resolution (or total pixels) of the frame R_k .

$$F(t) = \left(\sum_{k=t-4}^t \left| \frac{R(k)}{C(k)} - \frac{R(k-1)}{C(k-1)} \right| \right), \text{ where } C \text{ is the bandwidth or throughput} \quad (2)$$

4.3.3 Polynomial regression model for Quantization Parameter

Pixeltools describes how any codec dynamically adjusts encoder parameters to achieve a target bitrate, which is relevant for the video codec in the case of VCAs as well. Block-based hybrid video encoding schemes such as the MPEG and h.26* families are inherently lossy processes; this includes the VP9 codec used in Google Meet [3].

In particular, the quantization parameter QP regulates how much spatial detail is saved. When QP is very small, almost all that detail is retained. As QP is increased, some of that detail is aggregated so that the bitrate drops – but at the price of an increase in distortion and loss of quality. Figure 4 suggests that relationship for a particular input picture – if you want to lower the bitrate, you can do so by increasing QP at a cost of increased distortion. Figure 5 suggests that as source complexity varies during a sequence, you move from one such curve to another.

Thus, we try to model the QP (qpSum/s) or $P(t)$ from the bandwidth C and resolution R as it represents the frame size, and their ratio proportional to the bitrate, using polynomial regression for the same.

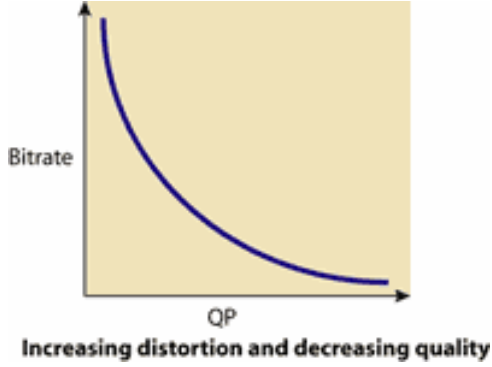


Figure 4: For a particular source frame.

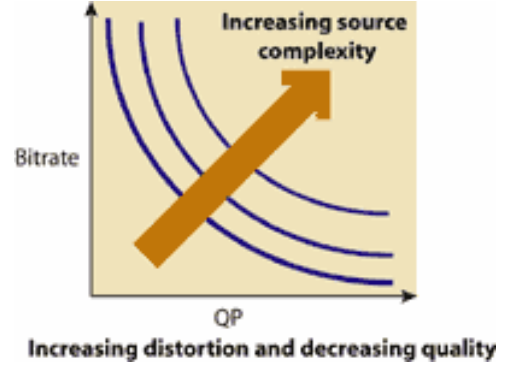


Figure 5: When source complexity varies.

$$P(t) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_0 = \sum_{k=0}^n a_k x^k \quad (3)$$

where x represents $C(t)/R(t)$ and coefficients a_i are model parameters.

4.4 Controller Architecture

The key idea of our MPC is to maximize QoE (expressed in terms of video quality terms) of users via a model predictive control method during the entire session. For each N -second horizon, MPC first uses a DL-based throughput predictor to estimate future bandwidth $[C_k, \dots, C_{k+N}]$ for the horizon. It then derives the frame jitter F_k and qpSum/s P_k from the predicted C_k . These are expressed in terms of unknown resolution R_k , which is obtained by maximizing the QoE, a normalized combination of these parameters, subject to dynamic constraints.

4.4.1 Why MPC?

To begin with, model predictive control may be naturally suitable for the problem of bitrate adaptation [7]. Although we cannot say that MPC is the best choice among all possible ABR algorithms, we can state that the MPC algorithm is more effective in mobile network video conferencing. Ideally, for a video call $[t_k, t_{k+N}]$, the QoE optimization problem can be directly addressed by the optimal resolution $R_1, \dots, R_k$, jitter, and QP.

However, in practice, we do not have access to this perfect information, making it difficult to optimize the solution online. Despite the fact that perfect information for the future as a whole is not available, it is possible to obtain reasonably accurate throughput predictions over a short span of $[t_k, t_{k+N}]$ in the near future. Owing to network conditions (and video call metrics) remaining relatively stable over short periods and not changing drastically within a few tens of seconds, we can exploit this property of throughput to execute QoE optimization. Following the receding horizon principle, we apply only the

first computed resolution R_k and shift the optimization horizon forward from step k to $k + 1$. This scheme is known as MPC. The advantage of MPC is that it can use prediction to optimize complex control objectives in dynamic systems online, subject to constraints.

4.4.2 MPC Controller

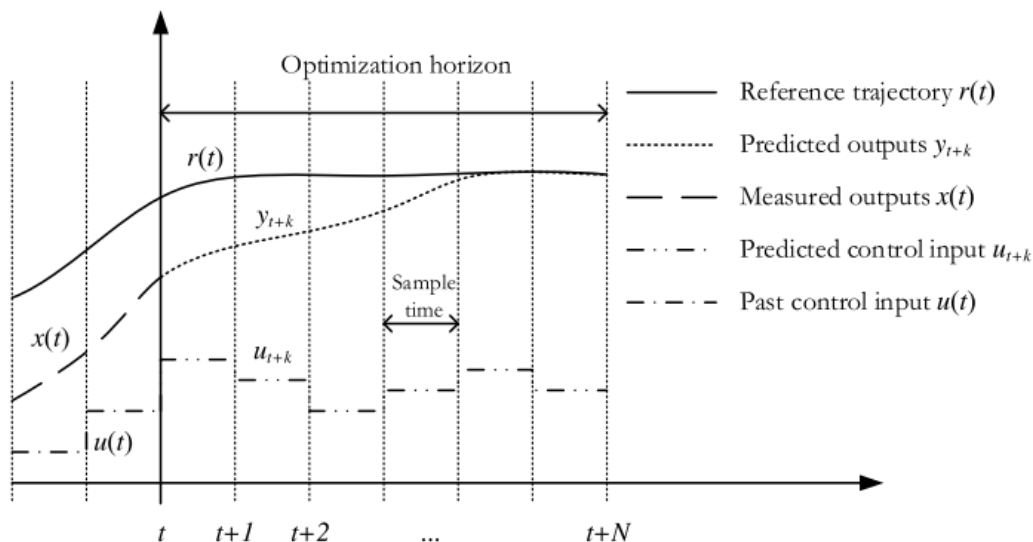


Figure 6: Inputs and outputs of the MPC-based controller.

Figure 6 shows the inputs and outputs of the MPC controller [11]. MPC uses a system model to predict the future states of the system and generates a control vector (Manipulated Variable (MV)) that minimizes a certain cost function over the prediction horizon in the presence of constraints. The first element of the computed control vector at any sampling instant is applied to the system input, and the remainder is discarded. The entire process is repeated at the next instant. The cost function can take the form of tracking error, control effort, frame jitter, low-resolution penalty, qpSum, or a combination of these factors.

Constraints can be placed on the rate and range limits of manipulated variables (e.g., upper and lower limits of jitter, qpSum, and range limits for resolution). This effort results in a controller that is robust to both time-varying disturbances and system parameters and regulates the process tightly within given bounds.

MPC performs the following three key steps: predict, optimize, and apply.

- **Predict:** We use the LSTM model to predict throughput across the look-ahead horizon. Using this predicted bandwidth, we derive the future trajectory values for the frame jitter $F(i)$ and QP $P(i)$ subsystems for each step i in the horizon,

expressing them as a function of the future manipulated variables, i.e., the planned resolutions $R(i)$.

- **Optimize:** This is the core of the MPC algorithm: Given the previous resolution $R[t_{k-M}, t_{k-1}]$, jitter $F[t_{k-M}, t_{k-1}]$, QP $P[t_{k-M}, t_{k-1}]$, and throughput prediction $C[t_k, t_{k+N}]$, find the optimal resolution R_k .

The selection is modeled as:

$$R_k = f_{\text{mpc}}(R[t_{k-M}, t_{k-1}], P[t_{k-M}, t_{k-1}], F[t_{k-M}, t_{k-1}], C[t_k, t_{k+N}]) \quad (4)$$

This is implemented by solving QoE_{max} . The function f_{mpc} also includes current jitter F_k and framerate P_k terms which are expressed using R_k as the input to their respective models.

- **Apply:** Use the $R[t_k, t_{k+N}]$ control vector as the resolution in the VCA for optimized QoE in the horizon $[t_k, t_{k+N}]$.

4.4.3 Problem Formulation

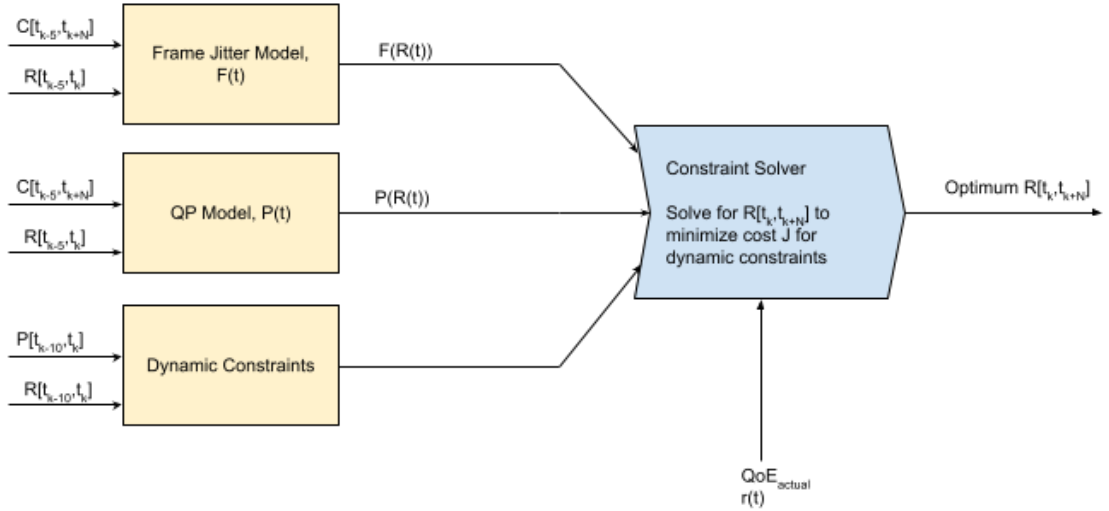


Figure 7: MPC Controller architecture.

In this section, we formally describe the optimization problem, give details of the constraints and cost function considered by our MPC controller, and explain how we embody the ABR algorithm into the MPC framework.

Consider a multi-input single-output (MISO) plant S , with input u and output y signals sampled at a regular time interval T_s [2]. We aim at synthesizing a controller C for S such that the controlled system achieves a desired engineering objective defined

in terms of minimization of a cost $J(y_{1:T}, \{u_{1:T}\})$, where $u_{1:T}$ denotes the sequence of input signals measured at time steps $t = 1, \dots, T$, and T is the length (measured in the number of samples) of the experiment where the performance is measured.

The inputs u_t to S in our problem will be frame jitter F , resolution R , and QP P . However, as mentioned in Section 5, since we model F and P , and can express them in terms of R , our MISO system gets simplified to a single-input single-output (SISO) system. The output y represents QoE.

Besides minimizing the cost $J(y_{1:T}, \{u_{1:T}\})$, the following domain constraints on inputs and outputs should be satisfied:

$$u_{\min} \leq u(t) \leq u_{\max} \quad (5a)$$

which for our problem is given as:

$$0 \leq F(t) \leq 1.5F_{\max}, \quad (5b)$$

$$R_{\min}/2 \leq R(t) \leq R_{\max} \quad (5c)$$

$$0 \leq P(t) \leq 1.5P_{\max} \quad (5d)$$

Constraints (5) impose the domain limitation for the inputs and give little room for the performance metric to worsen, since an optimization solution may exist wherein one or more inputs are worse but still provide the optimal QoE under given conditions. The control design problem is formulated as the following optimization problem:

$$\min_{R_k \in \mathcal{R}} J(y_{1:T}, u_{1:T}) \quad \text{s.t. (5)} \quad (6)$$

where the cost $J(y_{1:T}, \{u_{1:T}\})$ is the summation of mean squared error (Mean Squared Error (MSE)) over the horizon relative to a constant target reference $r(t) = 1$. Because the performance objective is normalized, this static reference represents the ideal, optimized QoE ceiling that the controller actively attempts to maintain.

$$J(y_{1:T}, u_{1:T}) = \frac{1}{N} \sum_{k=t}^{t+N} (y(k) - r(k))^2 \quad (7)$$

5 Evaluations

5.1 Throughput prediction using LSTM

In Figure 8, x represents the time, where negative values imply historic time when trying to predict the throughput at $t = 0$ and beyond, whereas y represents the normalized throughput, i.e., the number of outbound packets sent per second. The features are normalized to improve the quality of the data being fed to the model. We use an 80-20 split for the training and testing dataset. We were able to achieve an average Root Mean Square Error (RMSE) of 0.4 with this approach. This model was trained on the top 13 features of the dataset, as the remaining features had a negligible score.

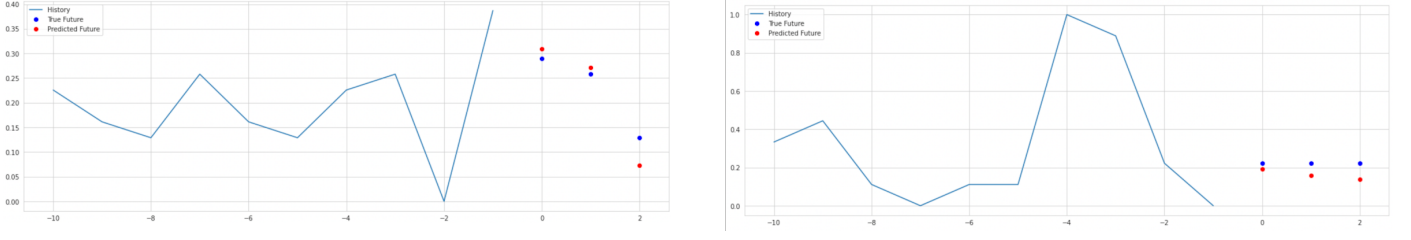


Figure 8: Throughput prediction using LSTM.

5.2 Throughput prediction using AutoML

We observed varying RMSE values for different datasets, ranging from 0.5 to a staggering 4. Note that the look-back window was chosen to be 5 seconds in contrast to the 10 seconds we had for the LSTM model. We let the FEDOT framework figure out the various hyperparameters to tune the model for optimal performance, as shown in Figure 9. The y -axis represents the throughput measured in terms of the number of outbound packets sent per second, while the x -axis represents the time in seconds.

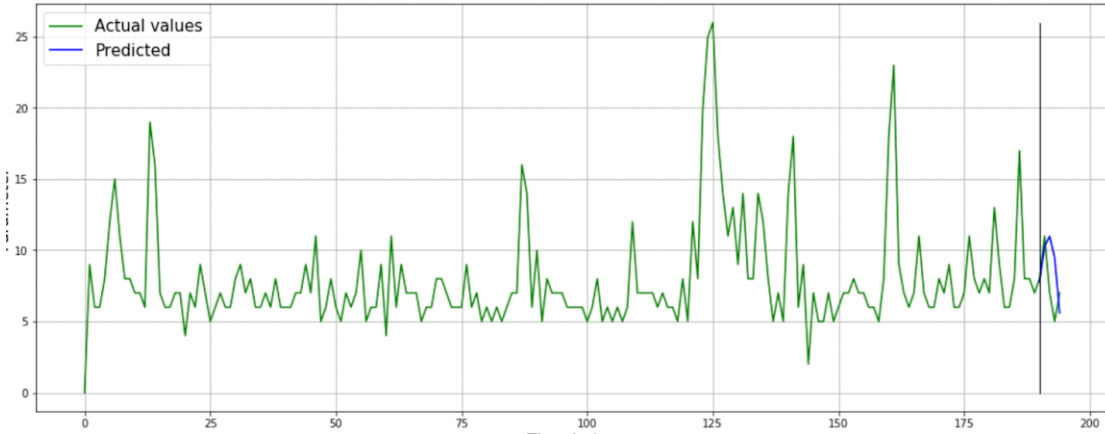


Figure 9: Throughput prediction using FEDOT

5.3 QP using Polynomial Regression

Section 4.3.3 explains an inverse relation between bitrate and QP. We use the `poly1d` class of the `numpy` module and employ polynomial regression to find a relation that fits well with our data, obtaining a polynomial relationship between bitrate (X) and QP (Y). Specifically, we refer to the ratio of bandwidth C and resolution R (as it represents the frame size) to represent the bitrate X . Additionally, the product of `qpSum/frameEncoded` and `framesEncoded` is taken as Y to get QP in per-second units like X . The result is a 3rd-degree polynomial given by coefficients (using $100X$ instead of X), detailed in Table 1:

Table 1: QP as a 3rd-degree polynomial function of bandwidth and resolution.

a3	a2	a1	a0
-659205.54713143	230130.0817032	-25840.53107608	1201.94775118

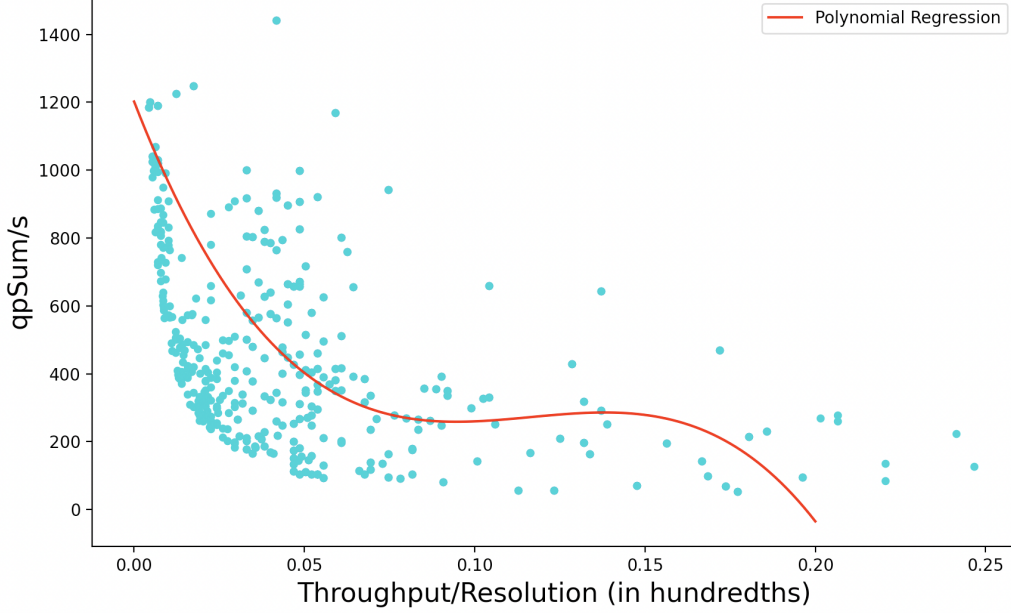


Figure 10: Inverse relation between qpSum and Throughput/Resolution underpins the eqns: $\Delta qpSum = \overline{QP} \times FPS \times \Delta t$; and $Throughput \approx \left(\frac{Resolution \times FPS}{QP^2} \right) \times X$, where X represents a video complexity coefficient, such as high motion vs. a static background)

5.4 JitterNot vs Google Meet’s Existing ABR

Figure 11 provides a comparison between JitterNot’s suggested resolution and Google Meet’s actual resolution for two random time instances while Figure 12 illustrates the corresponding QoE metric. We have captured and analyzed JitterNot’s MPC performance using 10 different test video call sessions and a horizon of 5 seconds, and summarized the gain in QoE that JitterNot achieves against using Google Meet in Table 2. Section 4.3 explains the QoE metric evaluation, and the details for frame jitter and quantization parameters are explained within its subsections.

In the chosen time windows, besides a single instance at $t = 1s$ in Figure 12, the QoE for our proposed work remains higher than the actual QoE for the same time instances. Figure 11, shows that the overall suggested resolution never degraded when compared to Google Meet.

However, since we optimized the system for our particular QoE metric, which might

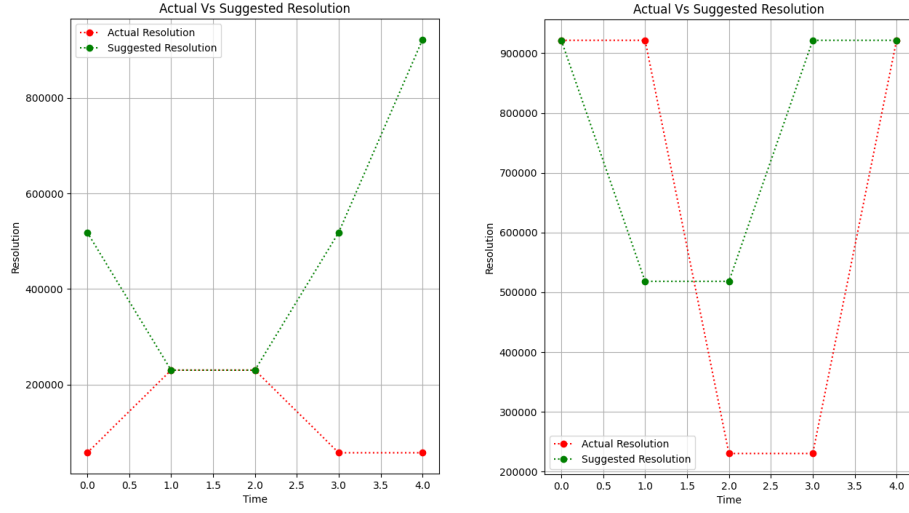


Figure 11: Comparison of JitterNot and Google Meet's resolutions

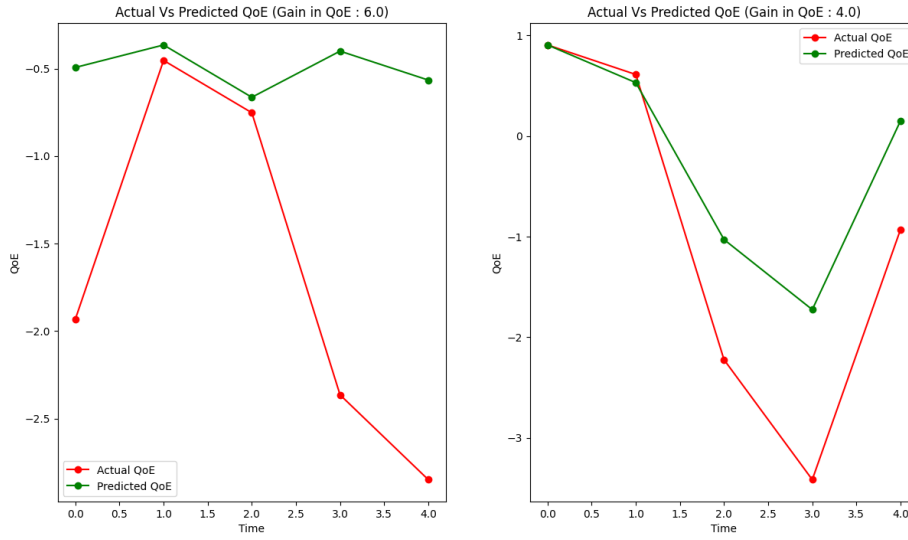


Figure 12: Comparison of JitterNot and Google Meet's QoE

differ from Google Meet's overall objectives, we also compare the video quality terms and summarize our findings in Table 2.

Table 2: Improvement in our QoE metric and video quality terms vs. Google Meet (in %).

Avg. Gain in QoE	Avg. Gain in $q(R)$	Avg. in Frame Jitter	Avg. Gain in QP
18.78	68.5	-18.97	9.77

6 Conclusion

In this work, we presented JitterNot, an innovative ABR approach for real-time video conferencing featuring the fusion of RNN and model-predictive control methods. JitterNot improves upon existing ABR approaches for VCAs, which often use fixed control rules or heuristics and consequently suffer from the limitation of ignoring the deployment environment and network conditions. The salient features of our closed-loop control system are: (i) an LSTM model that simplifies the system from MISO to SISO for the MPC (ii) the MPC controller that utilizes throughout predicted for the future horizon, and contextual information, namely the QP and frame jitter from the past window, to optimize the perceived QoE.

In our evaluations, we compared our approach to the default in Google Meet and found that JitterNot often achieved better QoE scores. Furthermore, it demonstrated better video quality and lower jitter, though it exhibited a slightly higher QP while transmitting frames. However, because our solution acted on top of another internal control loop inside the VCA’s video codec, an interaction loop interplay may have occurred. This dynamic interaction between the loops, if strong enough, could cause problems like longer response times and higher jitter, though it was not apparent during our evaluations. Since disabling the internal controller would solve the issue of loop interaction, we are optimistic that our solution might show superior performance in open-source VCAs that allow this configuration. Another concern is the decision-making execution latency that depends on the constraint solver, since timely action is crucial for maintaining a high QoE. On our experimental setup, it took 100–400 ms for every decision, which is high for a client-side solution. This could be addressed by delegating the computational solving tasks to an external server.

The results suggest that significant improvements can be made to the current techniques used by commercial VCAs by employing DL-based solutions since, unlike rule-based approaches, they can inherently account for fluctuating network conditions across different deployment environments and adapt to them dynamically.

References

- [1] “A beginner’s guide to video resolution”. In: *Adobe (Computer software company)* (2022; [available](#)).
- [2] S. Formentin D. Piga M. Forgione and A. Bemporad. “Performance-Oriented Model Learning for Data-Driven MPC Design”. In: *IEEE Control Systems Letters*,

vol. 3, no. 3, pp. 577-582, July 2019, doi: 10.1109/LCSYS.2019.2913347 (July 2019).

- [3] “Dynamically adjust encoder parameters to achieve a target bitrate”. In: *PixelTools (Video Captioning And Processing Experts)* (2015; [available](#)).
- [4] Sadjad Fouladi John Emmons, Emre Orbay Catherine Wu Riad S. Wahby, and Keith Winstein. “Salsify: Low-Latency Network Video”. In: *5th USENIX Symposium on Networked Systems Design and Implementation (NSDI '18)*, (April 9–11, 2018).
- [5] D. Ammar K. De Moor M. Xie M. Fiedler and P. Heegaard. “Video QoE killer and performance statistics in WebRTC-based video communication”. In: *2016 IEEE Sixth International Conference on Communications and Electronics (ICCE)*, 2016, pp. 429-436 (2016).
- [6] Mohammad Alizadeh Hongzi Mao Ravi Netravali. “Neural Adaptive Video Streaming with Pensieve”. In: *Proceedings of the 2017 ACM SIGCOMM Conference* (2017).
- [7] Y. Wang T. Zhou C. Hou B. Hou and Z. Cai. “2prong: Adaptive Video Streaming with DNN and MPC”. In: *2021 17th International Conference on Mobility, Sensing and Networking (MSN), 2021*, pp. 726-731, doi: 10.1109/MSN53354.2021.00114 (2021).
- [8] T. Huang and L. Sun. “Deepmpc: A Mixture Abr Approach Via Deep Learning And Mpc”. In: *020 IEEE International Conference on Image Processing (ICIP)*, 2020, pp. 1231-1235, doi: 10.1109/ICIP40778.2020.9191198 (2020).
- [9] Netravali Ravi Anirudh Sivaraman Somak Das Ameesh Goyal Keith Winstein James Mickens and Hari Balakrishnan. “Mahimahi: Accurate Record-and-Replay for HTTP”. In: *USENIX Annual Technical Conference (USENIX ATC 15)*, pp. 417-429., (2015).
- [10] Calvin Nguyen. “WebRTC — The technology that powers Google Meet, Medium”. In: (Mar 6, 2020; [available](#)).
- [11] Abdul Afram Farrokh Janabi-Sharifi Alan S. Fung Kaamran Raahemifar. “Artificial neural network (ANN) based model predictive control (MPC) and optimization of HVAC systems”. In: *Volume 141, 2017, Pages 96-113, ISSN 0378-7788; elsevier* (2017).
- [12] “RTCRemeoteOutboundRtpStreamStats”. In: *MDN Web Docs (Resources for Developers)* (Jun 1, 2022; [available](#)).
- [13] Nikunj Baid Satyam Awasthi Shubham Talbar. “JitterNot-Adaptive-Video-Streaming-in-VCAs-with-DNN-and-MPC”. In: *GitHub Repository for JitterNot* (June 2022; [repository](#)).
- [14] Yin Xiaoqi Abhishek Jindal Vyas Sekar and Bruno Sinopoli. “A control-theoretic approach for dynamic adaptive video streaming over HTTP”. In: *In Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, pp. 325-338. 2015 (2015).

- [15] “Why is Jitter an Important WebRTC Metric for Contact Centers?” In: *CallStats.io (The WebRTC call quality tool)* (January 21, 2019; [available](#)).